

به نام خدا

آزمایش 5 درس آزمایشگاه
طراحی سیستم های دیجیتال –
ضرب کننده booth با اندکی
تغییر

امیر محمد شربتی (402106112) – پوریا رحمانی (402111418)

در این آزمایش باید یک ضرب کننده Booth با یک ویژگی مضاعف طراحی کنیم: باید قابلیت اینکه بتوان در یک کلاک چند بیت شیفت داد را هم اضافه کنیم.

اول اینکه ضرب کننده بوث چیست؟ booth multiplier یک الگوریتم سخت افزاری است که برای ضرب دو عدد باینری علامت دار استفاده می شود. این روش تلاش میکند با کاهش تعداد partial product ها، سرعت ضرب را بالا ببرد.

در این ویدئو ضرب کننده booth با یک مثال توضیح داده شده است. بنده از همین ویدئو برای مرور این الگوریتم استفاده کردم: <https://www.youtube.com/watch?v=4zFd3zZlguY>

فرض می کنیم ورودی ها دو عدد n بیتی هستند. در این الگوریتم یک عدد $2n+1$ بیتی را در نظر می گیریم. نام آن را operator گذاشتیم. این عدد شامل سه بخش است. در ابتدا بیت سمت راست آن صفر است. n بیت وسط همان multiplier هستند که با Q در کد نشان داده شده است. n بیت سمت چپ هم accumulator هستند که در ابتدا صفر هستند.

حال یک عدد $2n + 1$ بیتی داریم. در هر مرحله به دو بیت سمت راست نگاه میکنیم (در الگوریتم booth پیشرفته به سه بیت یا حتی چهار بیت نگاه می کنیم.) اگر هر دو صفر یا هر دو یک بودند، کاری نمیکنیم، اگر به صورت 01 بودند، multiplicand (یا همان M در کد) را با n بیت سمت چپ جمع میکنیم. انگار M را $n+1$ به چپ شیفت داده و سپس با operator جمع می کنیم. اگر هم دو بیت سمت راست به صورت 10 باشند، multiplicand را کم میکنیم.

حالا خواسته سوال این است که وقتی عملیاتی انجام نمی دهیم (یعنی وقتی دو بیت سمت راست 00 یا 11 هستند)، تا جای ممکن عملیات شیفت را در یک کلاک انجام دهیم تا سرعت مدار ضرب کننده بیشتر شود.

افزون بر این توصیف آزمایش نگفته است که اعداد ورودی چند بیتی هستند. پس ما تلاش کردیم تا پیاده سازی کد وریلاگ ما در برگیرنده حالت کلی باشد. یعنی یک پارامتری در ورودی تعریف کردیم که بر مبنای آن میشد این مازول ها را به ازای هر n برای تعداد بیت های دو عدد ورودی استفاده کرد.

نکته دیگر سوال این است که controller و datapath در پیاده سازی مازول های مجزایی باشند.

پس از این مقدمات به پیاده سازی کد می پردازیم.

در این کد 5 ماژول داریم.

یک ماژول datapath، یک ماژول control unit، یک ماژول اصلی که متصل کننده این دو بلوک است، یک ماژول برای اینکه مکانیزم چند بیت شیفت دادن را پیاده سازی کنیم و یک ماژول تست.

برای اینکه بتوان قابلیت چند بیت شیفت دادن را پیاده سازی کرد، یک ماژول پیاده سازی کردیم تا اولین بیت متفاوت از راست را شناسایی کند. مثلاً اگر عدد 11000 داریم، این ماژول محل بیت 4 ام از راست را که 1 است (متفاوت با بیت های قبلی) پیدا میکند. بدین صورت متوجه می‌شویم که چند بیت می‌توان شیفت داد و مکانیزمی که سوال می‌خواهد برآورده می‌شود.

ماژول controller ساده است. البته فهم آن بسیار پیچیده است. (روی پیاده سازی آن بسیار فکر شد) ولی پیاده سازی ساده ای دارد. پس از اینکه به مقدار لازم شیفت انجام شد (به نظر دلیل خطای کد همین بخش است) بررسی می‌کنیم که بیت سمت راست 1 است یا صفر. چون مقدار شیفت انجام شده، پس اگر 1 باشد، یعنی ترکیب آنها 01 است، پس باید جمع انجام شود. و برعکس اگر صفر باشد، یعنی ترکیب 10 داریم و باید تفریق انجام شود.

ماژول datapath هم شامل جمع کننده و شیفت دهنده است. به روز رسانی مقدار operator در این قسمت انجام میشود.

برای اتصال این ماژول ها از booth_mult استفاده شد. این ماژول هم ساده است و صرفاً دو بلوک datapath و control_unit را متصل کرده است (از آنها instance گرفته است).

در نهایت برای تست مدار یک ماژول تست هم نوشته شد. اما متأسفانه (100 افسوس:!) دیدیم که پیاده سازی ایراد دارد.

بنده هرچقدر تلاش کردم موفق نشدم باگ را پیدا کنم. متأسفانه به خاطر کمبود وقت، موفق نشدم قبل از ارسال پیش گزارش باگ کد را رفع کرده و نتایج شبیه سازی و سنتز را ارائه دهم. انشاءالله این کار را برای آزمایش عملی در آزمایشگاه و تا هفته بعد که قرار باشد در آزمایشگاه جلسه برگزار شود، انجام می‌دهیم.